❑ Exercice 1 : *Évaluation d'expressions*

1. Pour chacune des expressions ci-dessous, si elle s'évalue sans erreur, indiquer son type et sa valeur. Sinon, indiquer la cause de l'erreur.
   a) `let t = 7 + 9;;`
   b) `let s = -11.6;;`
   c) `let at = '@';;`
   d) `let perimetre = 4 *. 2.5;;`
   e) `let v = 2.0**10;;`
   f) `let test1 = (0 < 7.5 && 7.5 < 42);;`
   g) `let r = print_int 12;;`
   h) `let coucou = "Bonjour " + "tout le monde";;`
   i) `let test2 = ("Avant" < "Après");;`
   j) `let test3 = true or false;;`
   k) `let peri = let cote = 5 in 4*cote;;`
   l) `let lang = let p = "OCaml" in p.[1];;`
   m) `let test4 = 7.5 > 0;;`
   n) `let hesitation = let b = "euh " in b^b^b^b;;`
2. Même question pour les expressions conditionnelles suivantes :
   a) `let a = if true then 2 else 2.5;;`
   b) `let a = if 2=1+1 then 'A' else 'B';;`
   c) `let a = if true then "ok";;`
   d) `let a = let b= -2.4 in if (b>0.) then b else -b;;`
   e) `let a = if true then print_int 2;;`

❑ Exercice 2 : *Calculs*

Écrire une expression dont l'évaluation donne :

1. 2^{20}
2. Le quotient dans la division euclidienne de 756 par 24
3. `true` si 12 564 est divisible par 13 et `false` sinon
4. $-2x + 7$ avec $x = -1.7$
5. Le plus petit entier représentable par un `int`

❑ Exercice 3 : *Définition de fonctions*

1. Écrire une expression permettant de définir les fonctions suivantes :
   a) $f : n \mapsto n^2 + 5$ (sur les entiers)
   b) $g : x \mapsto -12x + 3$ (sur les flottants)
   c) $h : (n, p) \mapsto \max(n, p)$ (sur les entiers)
   d) $i : (x, y) \mapsto \min(x, y)$ (sur les flottants)
   e) `delta` : $(a, b, c) \mapsto b^2 - 4ac$ (sur les flottants)
   f) `signe` qui à un entier n associe 1 si n est positif, 0 si n est nul et -1 si n est négatif
2. Écrire une expression permettant de définir les fonctions suivantes :
   a) Somme des n premiers entiers
   b) $n!$
   c) Somme des inverses des n premiers entiers strictement positifs
3. On rappelle que pour $a \in \mathbb{R}$ et $n \in \mathbb{N}^*$:

$$\begin{cases} a^n &= \left(a^{n/2}\right)^2 & \text{si } n \text{ est pair} \\ a^n &= \left(a^{(n-1)/2}\right)^2 \times a & \text{sinon} \end{cases}$$

   a) Définir une fonction récursive `puissance` sur les entiers correspondant à cette définition.
   b) En utilisant les fonctions de conversion entre entiers et flottants, définir une fonction puissance sur les entiers qui utilise l'opérateur `**` de puissance sur les flottants.
   c) Écrire et tester une fonction qui prend en argument deux entiers et vérifie que les deux versions de `puissance` (exponentiation rapide et conversion) renvoient bien le même résultat.

❑ Exercice 4 : *Types construits et filtrage par motif*

1. On souhaite modéliser des cartes à jouer à l'aide de types construits :
   a) Définir un type somme `couleur` ayant pour constructeurs `Coeur`, `Carreau`, `Pique` et `Trefle`.
   b) Définir un type somme `valeur` représentant les valeurs des cartes : `As`, `Valet`, `Dame`, `Roi` ou `Nombre of int` (pour les cartes de 2 à 10).
   c) Définir le type `carte` comme un couple : `type carte = valeur * couleur;;`
   d) À l'aide d'un filtrage par motif, écrire une fonction `points : carte -> int` qui renvoie le nombre de points d'une carte à la belote (hors atout) : 11 pour un As, 10 pour un 10, 4 pour un Roi, 3 pour une Dame, 2 pour un Valet, et 0 pour les autres cartes.
2. On souhaite représenter des formes géométriques planes :
   a) Définir un type somme `forme` avec deux constructeurs : un cercle `Cercle of float` (rayon) et un rectangle `Rectangle of (float * float)` (dimensions).
   b) Écrire une fonction `perimetre : forme -> float` calculant le périmètre d'une forme (on prendra $\pi \approx 3{,}14159$).
   c) Écrire une fonction `aire : forme -> float` calculant l'aire d'une forme.

❑ Exercice 5 : *Inférence de types et polymorphisme*

1. Sans utiliser l'environnement interactif, donner le type inféré par OCaml pour chacune des fonctions suivantes :
   a) `let f1 x y = (y, x);;`
   b) `let f2 f x = f (f x);;`
   c) `let f3 f g x = f (g x);;`
   d) `let f4 (b, x, y) = if b then x else y;;`
   e) `let f5 x = [x];;`
2. Inversement, donner la définition d'une fonction valide (sans boucle infinie ni levée d'exception) correspondant à chacune des signatures suivantes :
   a) `'a * 'b -> 'b * 'a`
   b) `'a -> 'b -> 'a`
   c) `('a -> 'b) -> ('b -> 'c) -> ('a -> 'c)`
   d) `('a -> 'b) -> 'a * 'a -> 'b * 'b`

❑ Exercice 6 : *Fonctions anonymes et ordre supérieur*

1. Écrire une fonction `appliquer_n : int -> ('a -> 'a) -> 'a -> 'a` qui prend en argument un entier $n \geq 0$, une fonction f et une valeur x, et renvoie $f^n(x)$ (l'application répétée n fois de f à x).
2. À l'aide de la fonction `appliquer_n` et d'une fonction anonyme, écrire une expression en une ligne calculant 2^{10}.
3. Écrire une fonction de composition `composer : ('b -> 'c) -> ('a -> 'b) -> ('a -> 'c)` qui renvoie la fonction $f \circ g$ sous forme d'une fonction anonyme.
4. Écrire une fonction d'ordre supérieur `derivee` prenant une fonction $f : \mathbb{R} \to \mathbb{R}$ et un pas $dx > 0$ et renvoyant la fonction anonyme $x \mapsto \frac{f(x+dx)-f(x)}{dx}$. Préciser sa signature.
5. On souhaite approcher la racine d'une fonction continue par dichotomie :
   a) Écrire une fonction d'ordre supérieur `dichotomie` prenant en argument une fonction continue f, deux réels a et b vérifiant $f(a) \cdot f(b) \leq 0$ et une tolérance $\varepsilon > 0$, et renvoyant une approximation du zéro de f sur $[a, b]$. Préciser sa signature.
   b) En déduire une approximation de $\sqrt{2}$ à 10^{-5} près en appliquant `dichotomie` à une fonction anonyme sur l'intervalle $[1{,}0\,;\,2{,}0]$.

❑ Exercice 7 : *Les listes et la récursion structurelle*

1. Sans utiliser les fonctions du module prédéfini `List`, écrire les fonctions récursives suivantes sur les listes à l'aide du filtrage par motif (constructeurs `[]` et `::`) :
   a) `longueur : 'a list -> int` qui calcule la longueur d'une liste.
   b) `somme : int list -> int` qui calcule la somme des éléments d'une liste d'entiers (la somme de la liste vide valant 0).
   c) `appartient : 'a -> 'a list -> bool` qui teste si un élément est présent dans une liste.

d) `n_ieme : int -> 'a list -> 'a` qui renvoie l'élément situé à l'indice n d'une liste ($n \geq 0$, le premier élément étant à l'indice 0). Si n est hors limites, on pourra lever une erreur avec `failwith "Indice invalide"`.

2. Écrire une fonction récursive `intervalle : int -> int -> int list` telle que `intervalle a b` renvoie la liste des entiers consécutifs de a à b inclus (ou la liste vide si $a > b$). Par exemple, `intervalle 3 7` renvoie `[3; 4; 5; 6; 7]`.
3. Inversion d'une liste :
   a) Écrire une fonction récursive naïve `renverser : 'a list -> 'a list` utilisant l'opérateur de concaténation `@`. Justifier que sa complexité temporelle est quadratique en la taille n de la liste.
   b) Pour obtenir une complexité linéaire en temps, écrire une fonction auxiliaire récursive terminale `rev_acc : 'a list -> 'a list -> 'a list` utilisant un accumulateur, puis en déduire une fonction `renverser_rapide : 'a list -> 'a list`.
4. Écrire une fonction récursive `partitionner : ('a -> bool) -> 'a list -> 'a list * 'a list` qui sépare une liste en un couple de deux listes : celle des éléments vérifiant le prédicat, et celle des éléments ne le vérifiant pas.

❑ **Exercice 8** : *Fonctions d'ordre supérieur sur les listes et type option*

1. En utilisant les fonctions `List.map` et/ou `List.filter` de la bibliothèque standard, écrire en une seule ligne (sans récursion explicite) :
   a) La fonction `carres : int list -> int list` qui renvoie la liste des carrés des entiers d'une liste.
   b) La fonction `pairs : int list -> int list` qui ne conserve que les entiers pairs d'une liste.
   c) La fonction `carres_pairs : int list -> int list` qui renvoie la liste des carrés des seuls entiers pairs d'une liste.
2. La fonction `List.fold_left : ('a -> 'b -> 'a) -> 'a -> 'b list -> 'a` applique un opérateur binaire le long d'une liste, de la gauche vers la droite, à partir d'une valeur initiale :
   a) À l'aide de `List.fold_left`, calculer en une ligne le produit des éléments d'une liste d'entiers.
   b) Réécrire la fonction `longueur : 'a list -> int` uniquement à l'aide de `List.fold_left`.
3. Le type prédéfini polymorphe `'a option` (valant `None` ou `Some v`) permet de sécuriser les fonctions partielles sans lever d'exception :
   a) Écrire une fonction `tete_opt : 'a list -> 'a option` qui renvoie `Some t` si la liste a pour tête t, et `None` si la liste est vide.
   b) Écrire une fonction `div_opt : int -> int -> int option` qui effectue la division entière de a par b et renvoie `None` si $b = 0$.
   c) Écrire une fonction `chercher_premier : ('a -> bool) -> 'a list -> 'a option` qui renvoie sous forme d'option le premier élément d'une liste vérifiant le prédicat donné, ou `None` s'il n'en existe aucun.